Name: Ronit Sahoo
Class: D20C
Roll No: 29
Batch: B

BLOCKCHAIN EXPERIMENT NO: 5

Aim - Hands-on Solidity programming assignments for creating smart contracts using the Remix IDE.

Theory:

1. Primitive Data Types, Variables and Functions (pure, view)

In Solidity, primitive data types form the foundation of smart contract development. The commonly used types are:

- **uint / int -** unsigned and signed integers of different sizes, for example uint256 and int128.
- **bool -** represents logical values, either true or false.
- **address -** holds a 20-byte Ethereum account address, typically used to store user accounts or contract addresses.
- **bytes / string -** store binary data and textual data respectively.

Variables in Solidity fall into three categories. State variables are stored permanently on the blockchain, local variables are temporary and exist only while a function is executing, and global variables are special predefined values supplied by the EVM such as msg.sender, msg.value and block.timestamp.

Functions carry the contract logic, and two special classifications exist:

- **pure -** cannot read or modify blockchain state and works only with its inputs and internal computation.
- **view -** may read state variables but cannot alter them.

This classification helps optimize gas usage and enforces function integrity, since calling a pure or view function externally costs no gas at all.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs let users or other contracts pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer succeeded. Solidity also supports named return variables, which improve readability and make debugging easier.

3. Visibility, Modifiers and Constructors

Function visibility defines who is allowed to call a function:

- **public -** available both inside and outside the contract.
- **private -** accessible only within the same contract.
- **internal -** accessible within the contract and any contracts derived from it.
- **external -** can be called only by external accounts or other contracts.

Modifiers are reusable blocks of code that change the behaviour of a function. They are most often used for access control, for example an onlyOwner modifier that restricts sensitive functions to the contract owner.

Constructors are special functions that execute exactly once, at the moment the contract is deployed. They initialize important values, such as recording the deploying account as the owner of the contract.

4. Control Flow: if-else and Loops

Control flow in Solidity works much as it does in conventional programming languages:

- **if-else -** allows conditional decision making inside the contract logic, for example checking that a balance is sufficient before transferring funds.
- **Loops (for, while, do-while) -** enable repeated execution of code, such as iterating over an array of users. Loops must be used carefully, because every iteration consumes additional gas and an unbounded loop can make a contract prohibitively expensive or even impossible to execute.

5. Data Structures: Arrays, Mappings, Structs and Enums

- **Arrays -** can be fixed size or dynamic and store an ordered list of elements, for example an array of addresses holding registered users.
- **Mappings -** store key-value pairs and allow very fast lookups, for example mapping(address => uint) for balances. Unlike arrays, mappings cannot be iterated over.
- **Structs -** group related properties into a single custom type, for example struct Player { string name; uint score; }.
- **Enums -** define a fixed set of named constants, which makes contract state far more readable, for example enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three data locations for storing variables:

- **storage -** data written permanently to the blockchain, which is where state variables live. It is the most expensive location in gas terms.
- **memory -** temporary storage that exists only for the duration of a function call, used for local variables and function inputs.
- **calldata -** a read-only, non-persistent area used for external function parameters. It is the most gas-efficient of the three because the data is never copied.

Understanding data locations matters because the choice directly affects both gas cost and contract performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei -** Ether is the native currency of Ethereum, and Wei is its smallest unit, with 1 Ether equal to 10^18 Wei. Working internally in Wei keeps financial calculations precise, since Solidity has no floating point type.
- **Gas and Gas Price -** every transaction consumes gas, which measures computational effort. The gas price sets how much Ether is paid per unit of gas, and a higher price gives validators an incentive to include the transaction sooner.
- **Sending Transactions -** transactions transfer Ether or interact with contracts. transfer() and send() are the older mechanisms, while call() is the currently recommended approach because it forwards all available gas and returns a success flag that must be checked.

Program & Output:

All assignments were completed in the Remix IDE using the Learneth "Solidity Beginner Course" module, compiled with Solidity 0.8.x and deployed on the Remix VM (Cancun). Each screenshot below shows the completed assignment together with the "Well done! No errors." confirmation returned by the checker.

Tutorial 1: Introduction

A Counter contract was written with a public state variable count, a view function get() to read it, and two state-changing functions inc() and dec() to increment and decrement it. The contract was then compiled and deployed on the Remix VM.

Output:

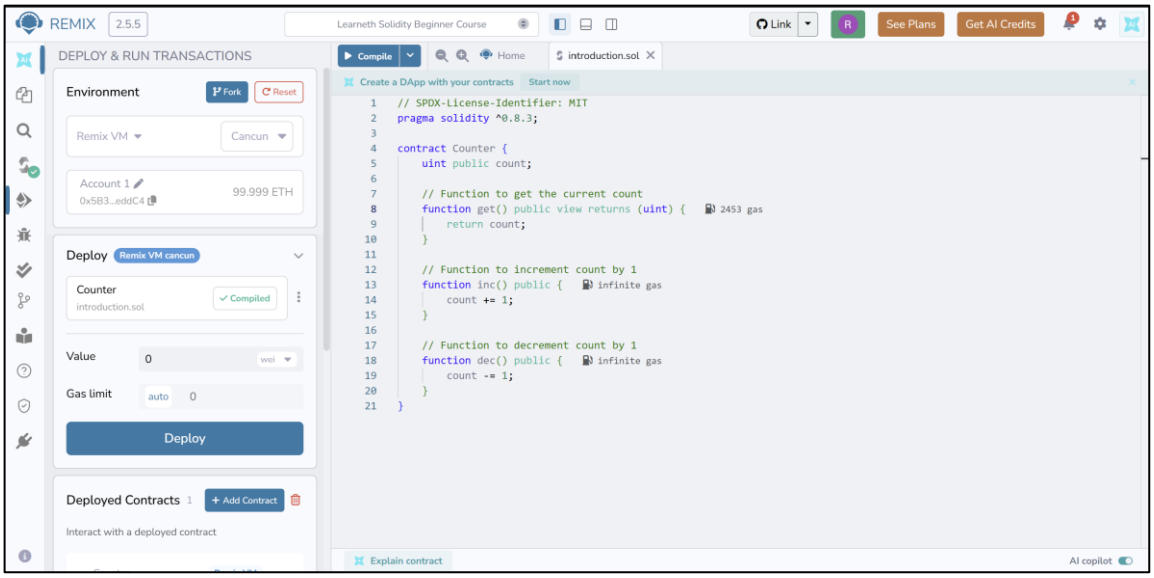


Fig 1.1: The Counter contract compiled and ready to deploy on the Remix VM.

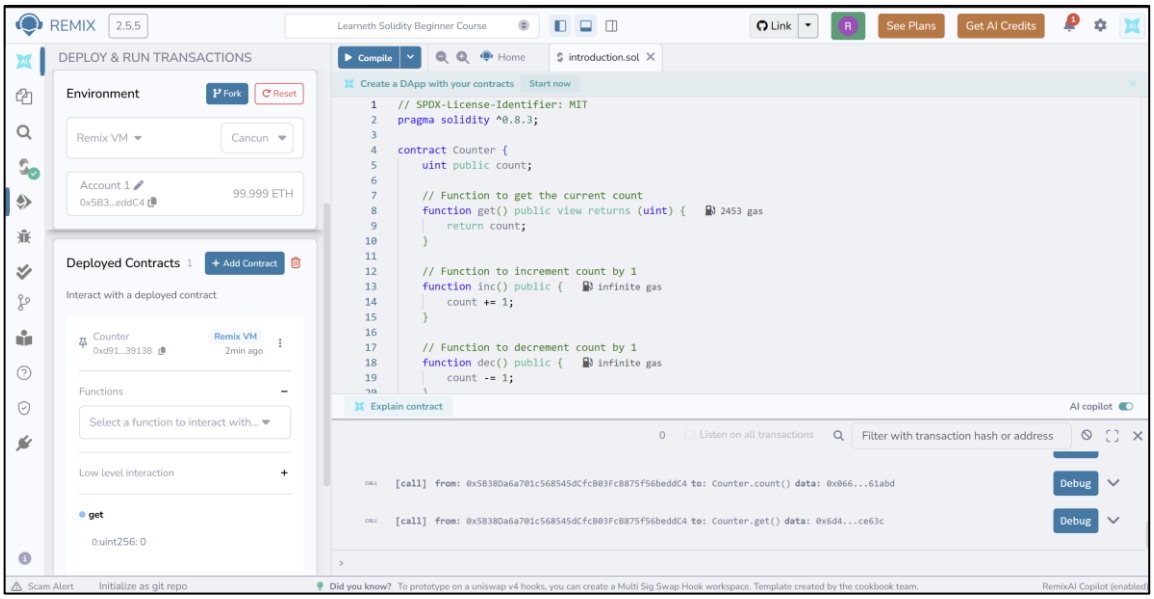


Fig 1.2: The deployed contract exposed under Deployed Contracts, with its call log below.

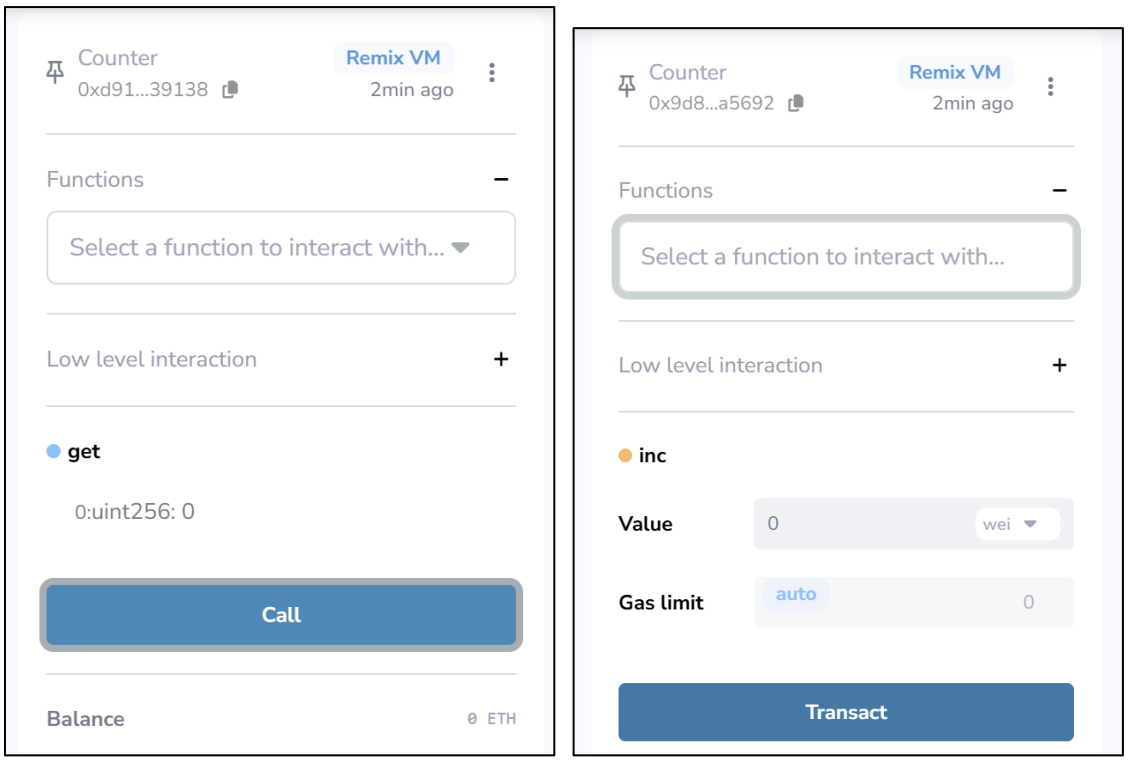


Fig 1.3: get() returning the initial value 0 (left) and the inc() transaction ready to be sent (right).

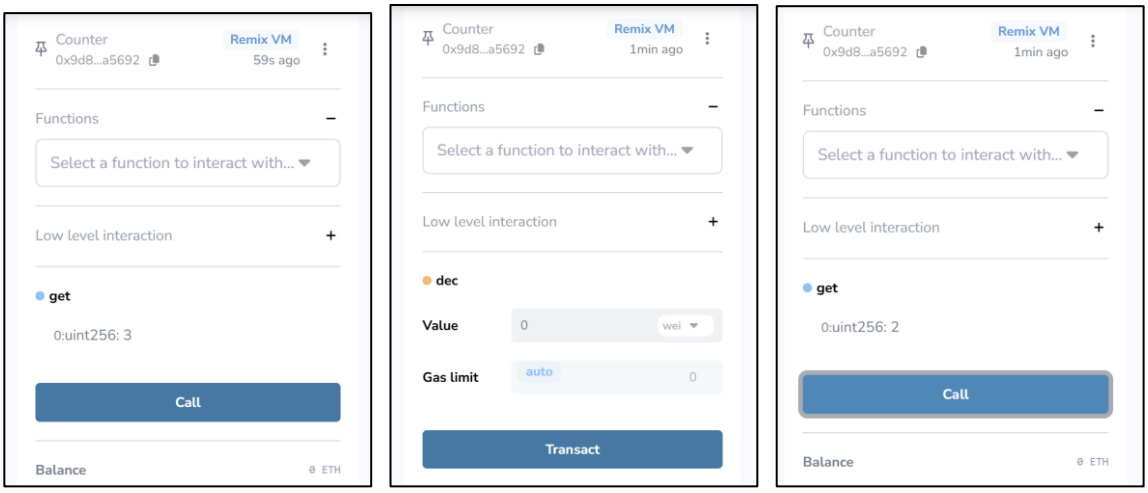


Fig 1.4: get() returning 3 after three increments (left), the dec() transaction (centre), and get() returning 2 afterwards (right).

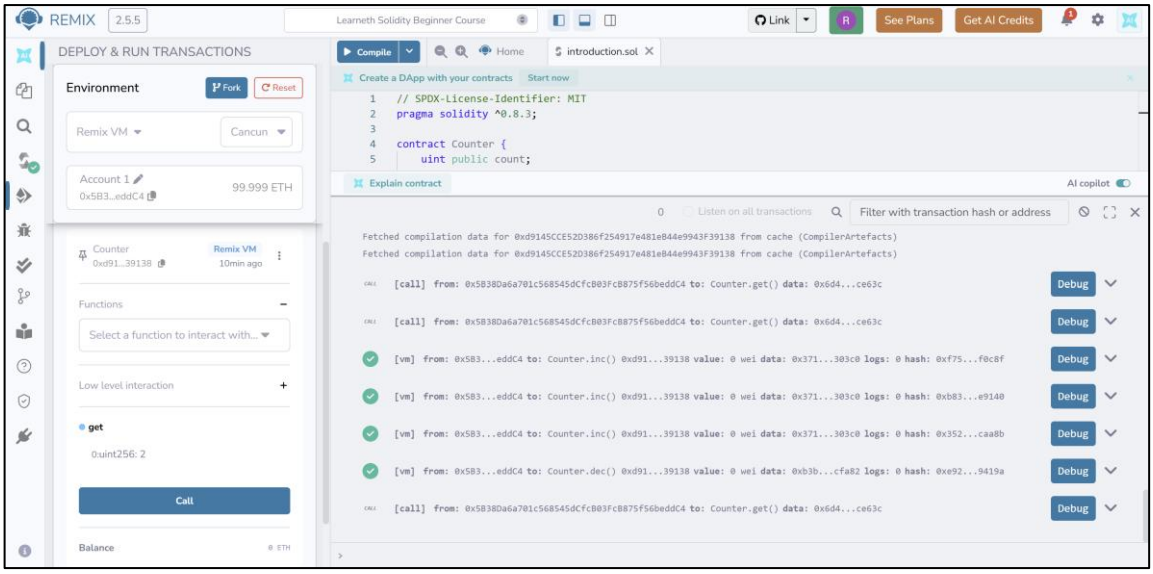


Fig 1.5: Full transaction log showing three inc() calls, one dec() call and the final get() result of 2.

Tutorial 2: Basic Syntax

The HelloWorld contract was replaced with a new contract named MyContract containing a public state variable name of type string, assigned the value "Alice".

Output:

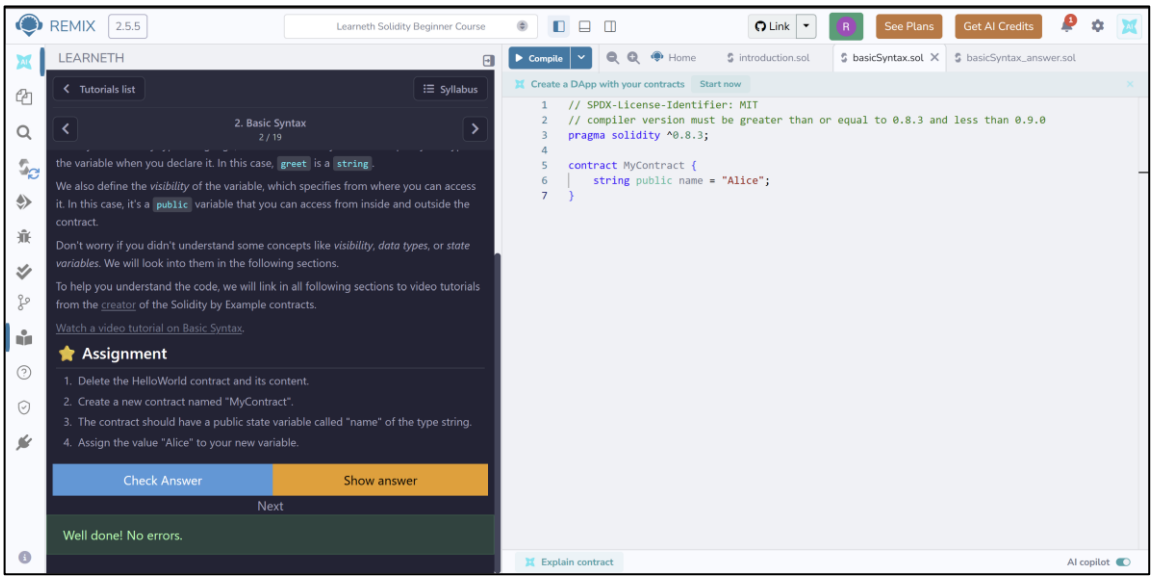


Fig 2: Basic Syntax - assignment completed and verified in Remix.

Tutorial 3: Primitive Data Types

A new public address variable newAddr, a public negative integer neg, and newU declared with the smallest available uint type were added to the contract.

Output:

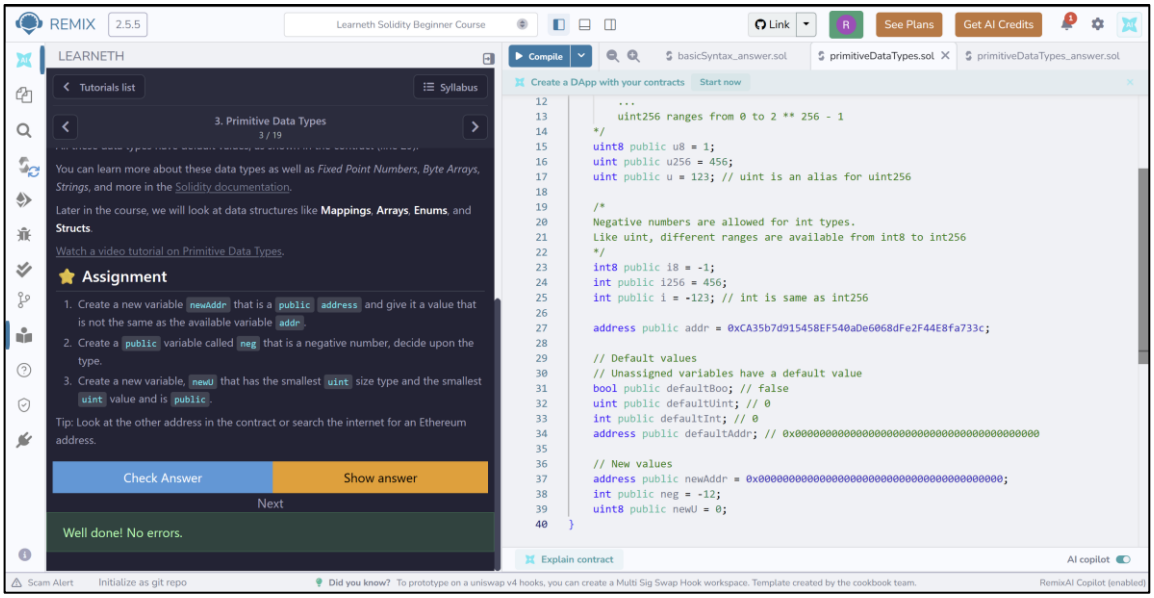


Fig 3: Primitive Data Types - assignment completed and verified in Remix.

Tutorial 4: Variables

State, local and global variables were declared and distinguished, showing which values persist on chain and which exist only for the duration of a call.

Output:

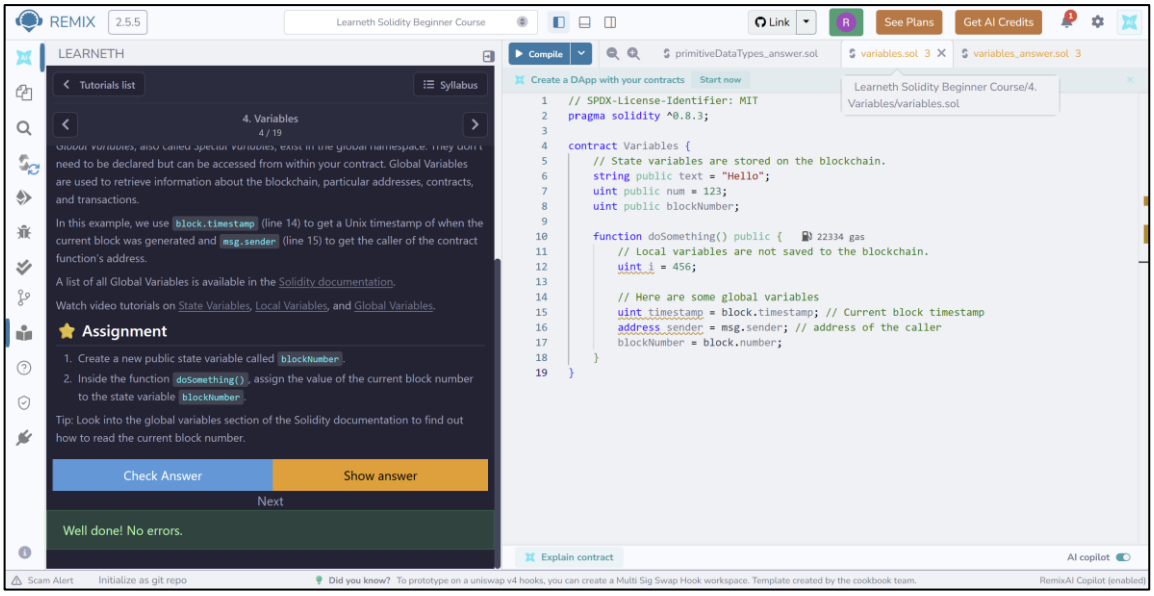


Fig 4: Variables - assignment completed and verified in Remix.

Tutorial 5: Functions - Reading and Writing to a State Variable

A function was written to write a value into a state variable and a matching view function to read it back, demonstrating the gas difference between the two.

Output:

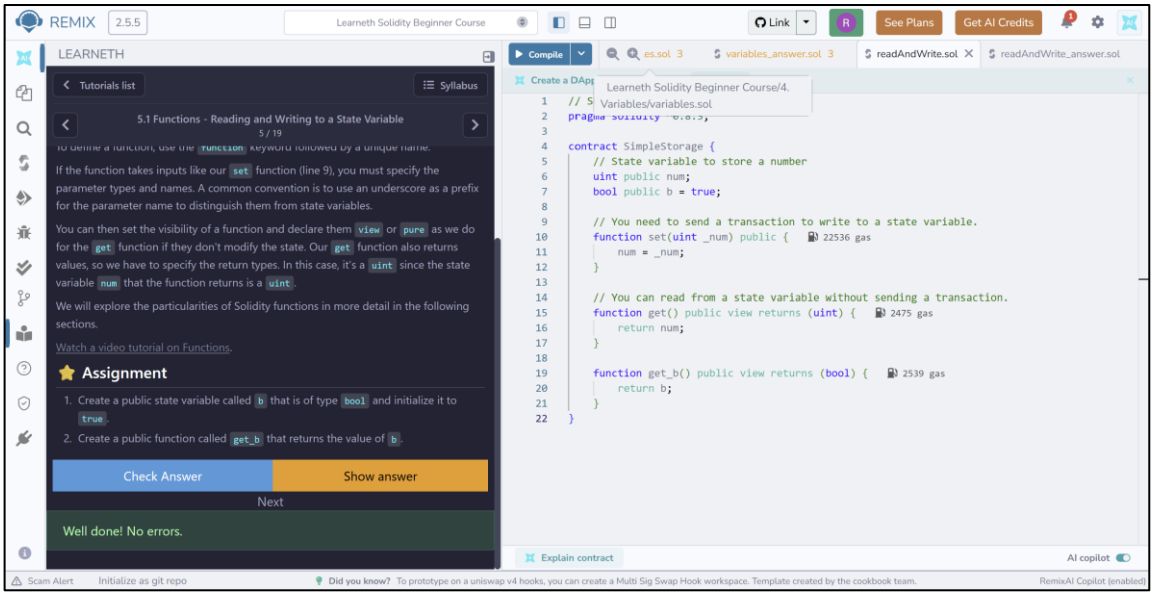


Fig 5: Functions - Reading and Writing to a State Variable - assignment completed and verified in Remix.

Tutorial 6: Functions - Modifiers and Constructors

A constructor was added to record the deployer as the owner, along with a modifier that restricts a sensitive function to that owner.

Output:

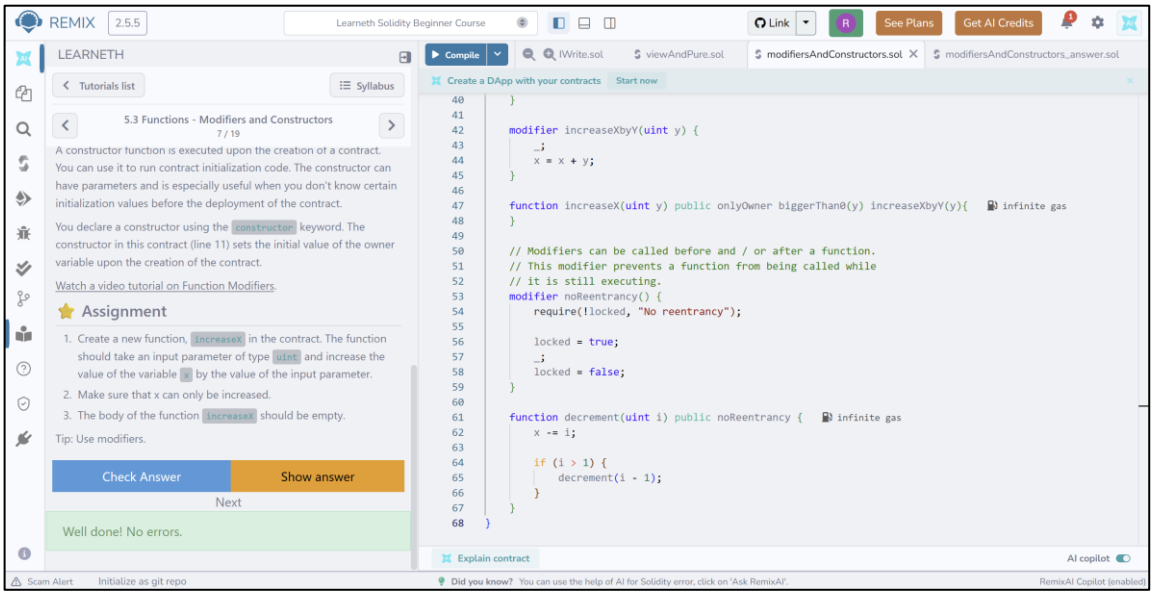


Fig 6: Functions - Modifiers and Constructors - assignment completed and verified in Remix.

Tutorial 7: Functions - Inputs and Outputs

Functions were written to accept input parameters and return output values, including the use of named return variables.

Output:

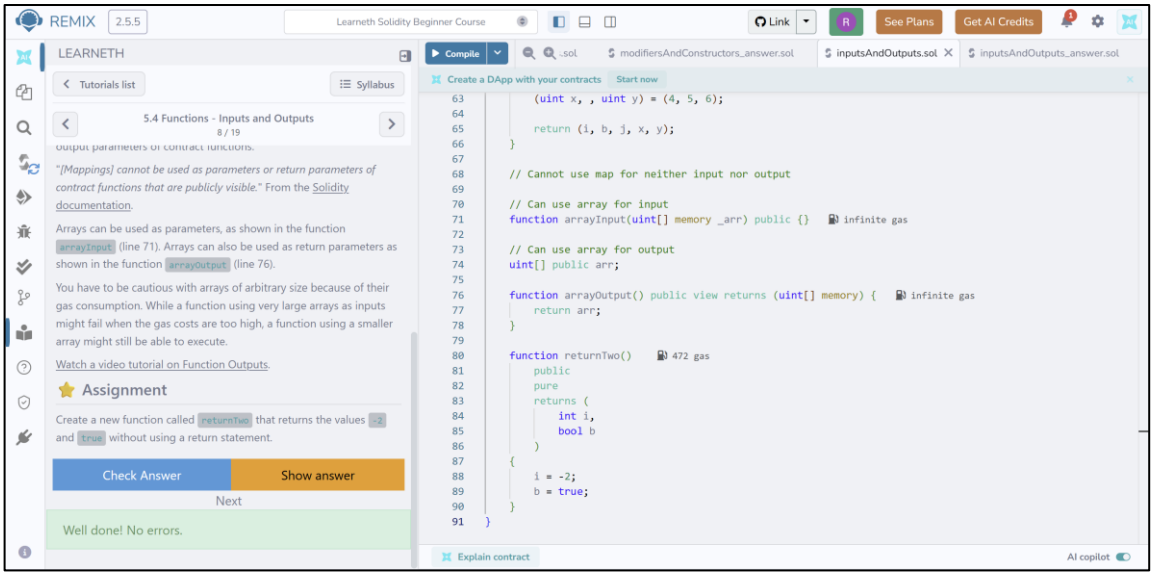


Fig 7: Functions - Inputs and Outputs - assignment completed and verified in Remix.

Tutorial 8: Visibility

The public, private, internal and external visibility specifiers were applied and their access restrictions verified from inside and outside the contract.

Output:

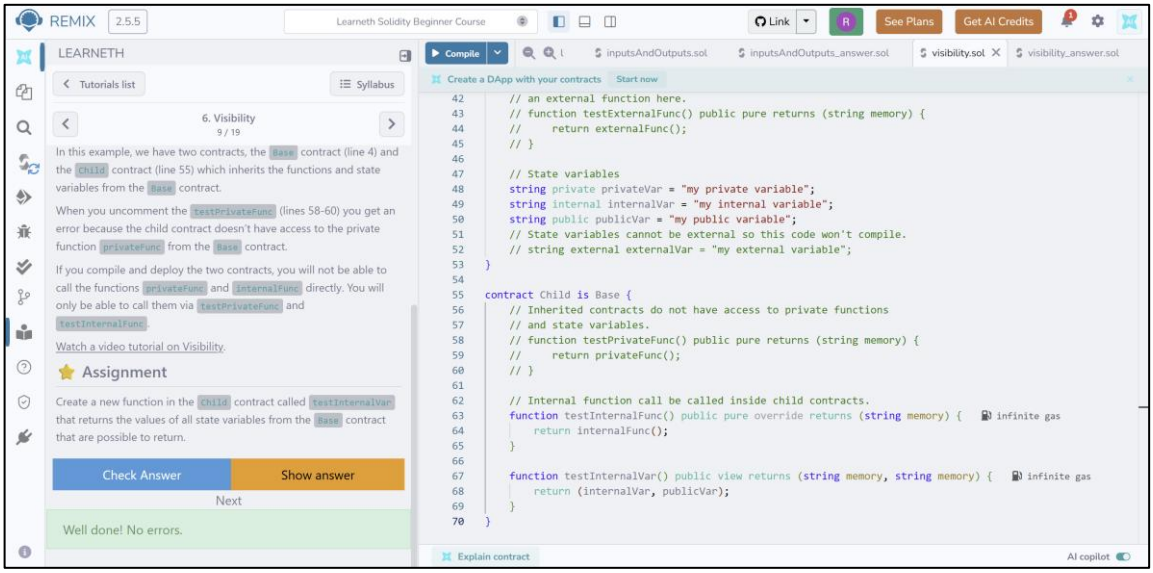


Fig 8: Visibility - assignment completed and verified in Remix.

Tutorial 9: Control Flow - If/Else

Conditional logic was implemented using if-else statements to branch on the value of a function input.

Output:

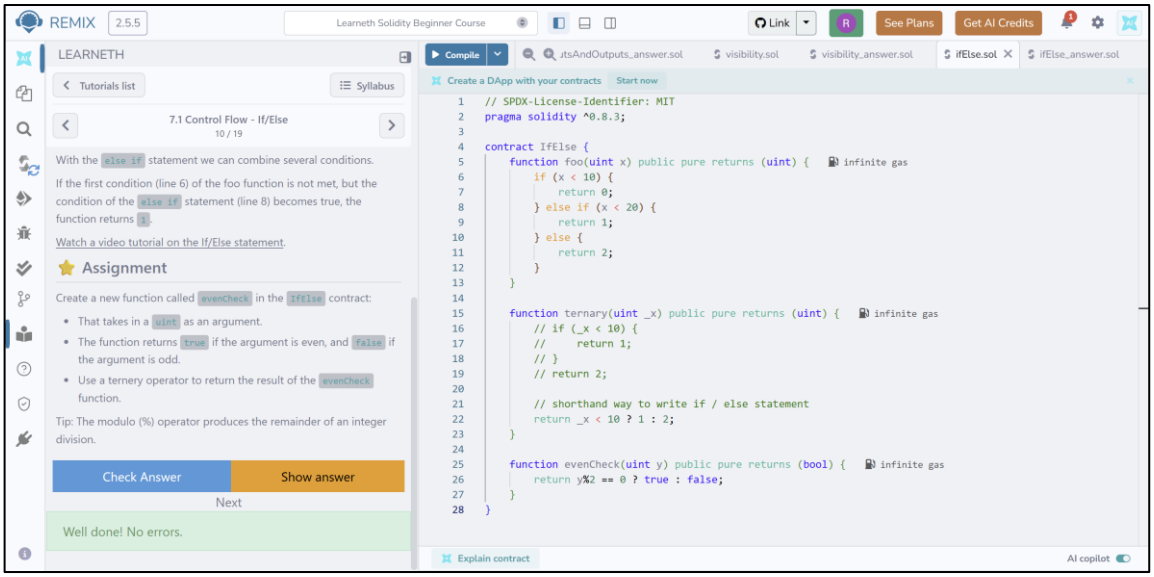


Fig 9: Control Flow - If/Else - assignment completed and verified in Remix.

Tutorial 10: Control Flow - Loops

for and while loops were used to iterate over data, with attention to keeping the iteration count bounded so gas consumption stays predictable.

Output:

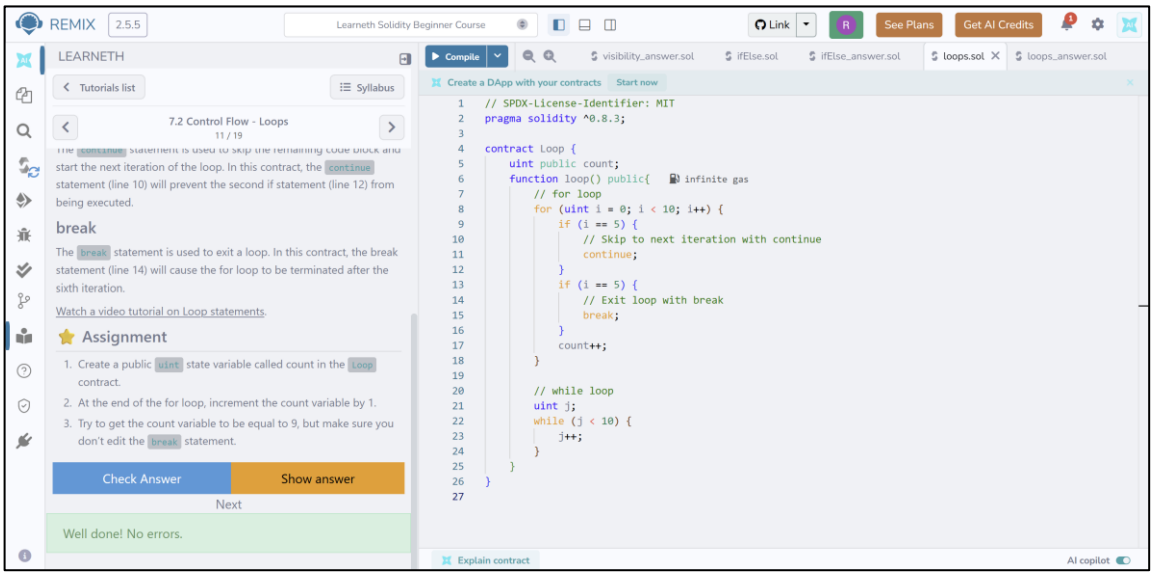


Fig 10: Control Flow - Loops - assignment completed and verified in Remix.

Tutorial 11: Data Structures - Arrays

Fixed-size and dynamic arrays were declared, and elements were pushed, read and removed.

Output:

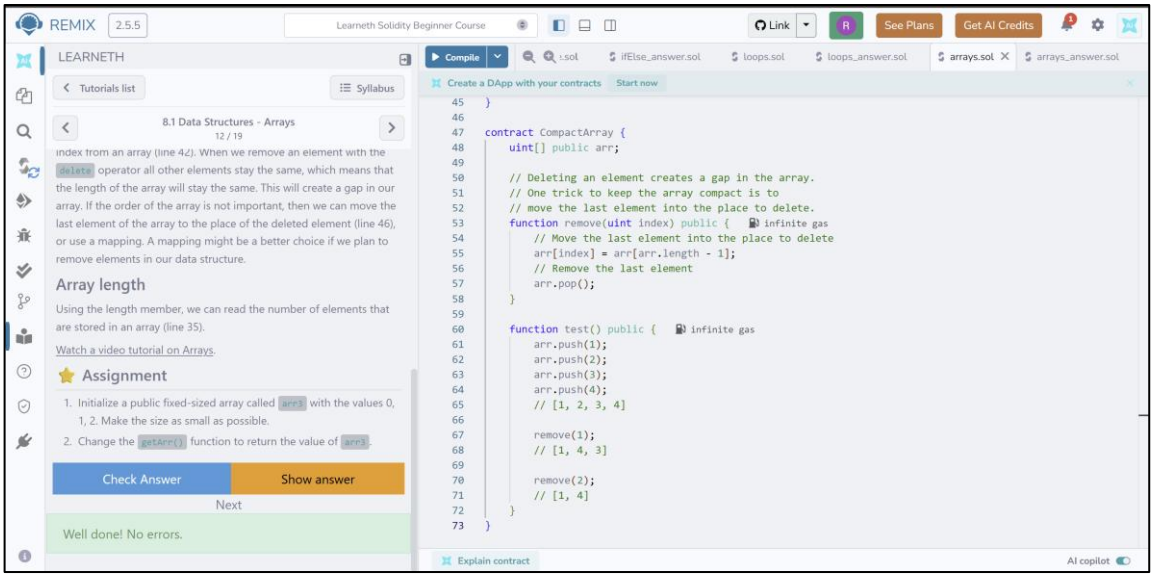


Fig 11: Data Structures - Arrays - assignment completed and verified in Remix.

Tutorial 12: Data Structures - Mappings

A mapping was used to store and retrieve values by key, demonstrating constant-time lookup and the fact that mappings cannot be iterated.

Output:

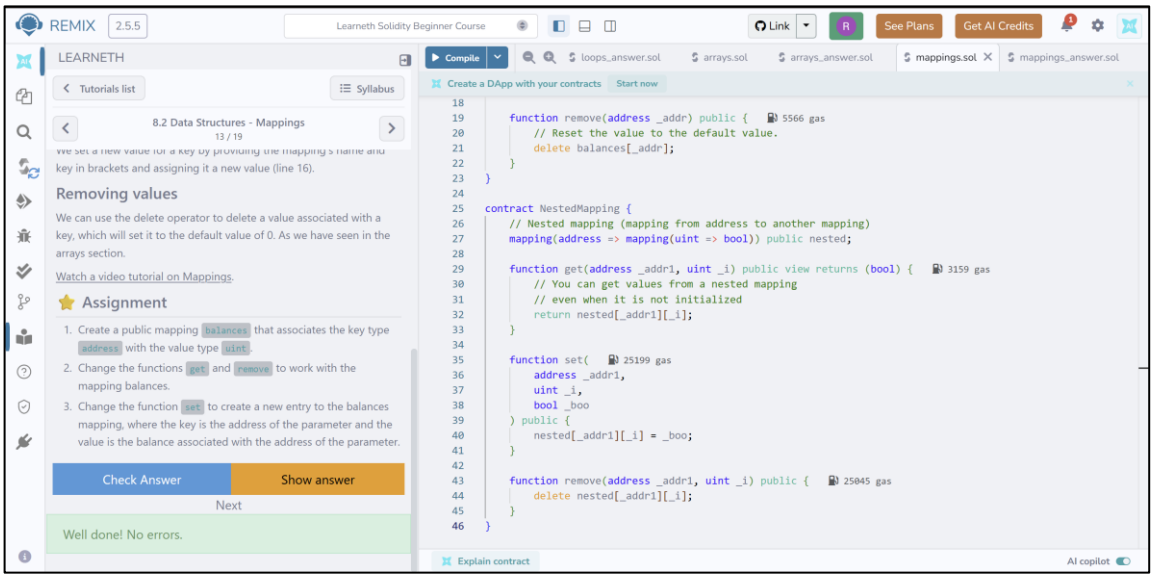


Fig 12: Data Structures - Mappings - assignment completed and verified in Remix.

Tutorial 13: Data Structures - Structs

A custom struct was defined to group related fields into a single type, then instantiated and stored on chain.

Output:

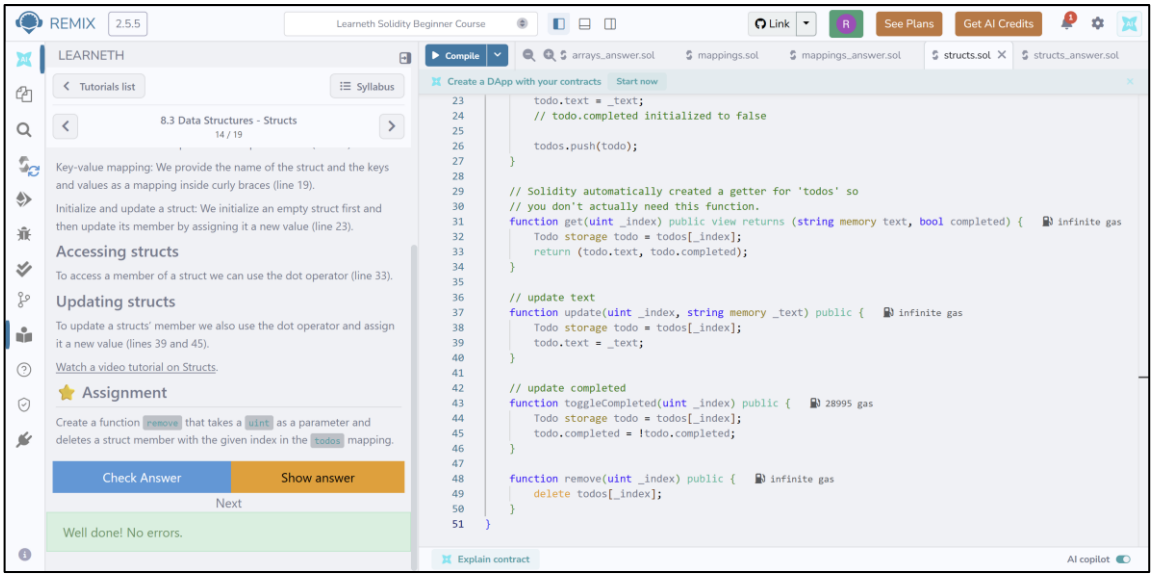


Fig 13: Data Structures - Structs - assignment completed and verified in Remix.

Tutorial 14: Data Structures - Enums

An enum was defined to represent a fixed set of contract states, and functions were written to read and update the current state.

Output:

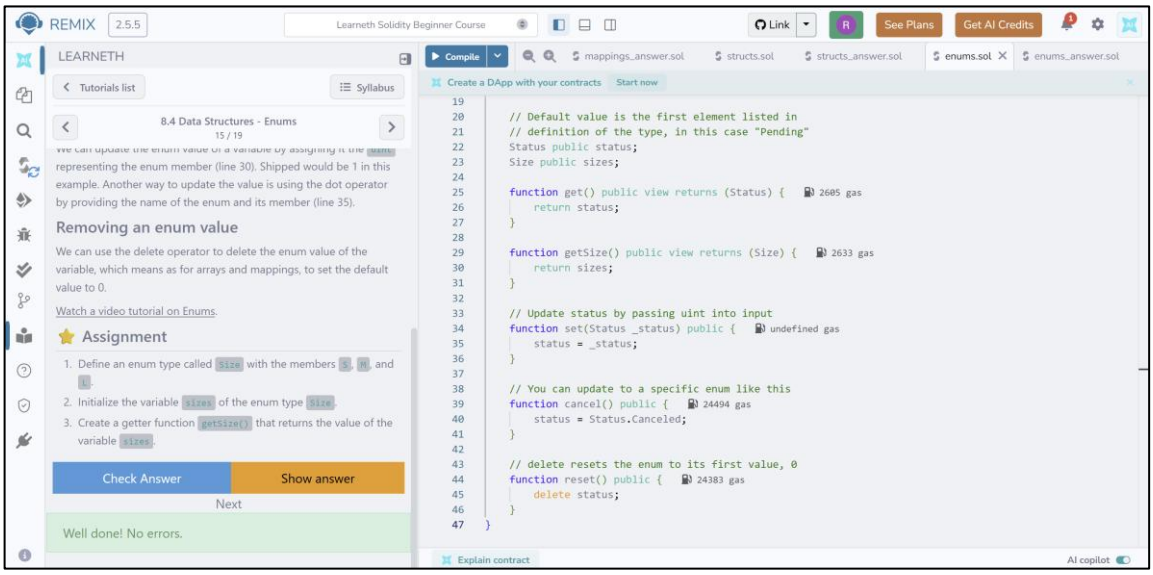


Fig 14: Data Structures - Enums - assignment completed and verified in Remix.

Tutorial 15: Data Locations

The storage, memory and calldata locations were used explicitly, showing where each is permitted and how the choice affects gas cost.

Output:

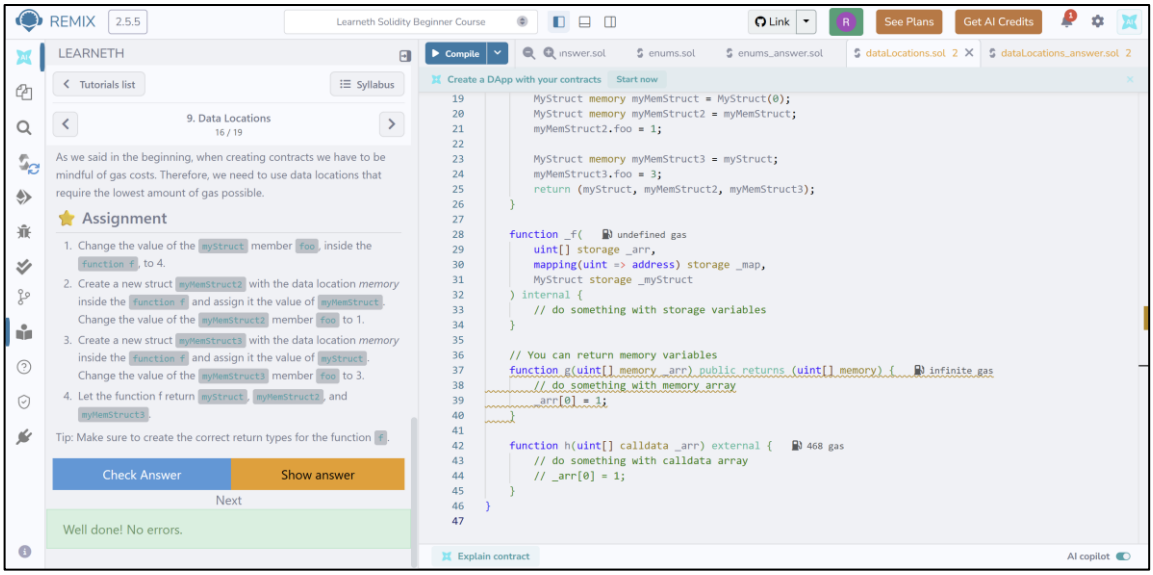


Fig 15: Data Locations - assignment completed and verified in Remix.

Tutorial 16: Transactions - Ether and Wei

Values were expressed in Wei, Gwei and Ether, confirming the 10^18 conversion factor between Ether and Wei.

Output:

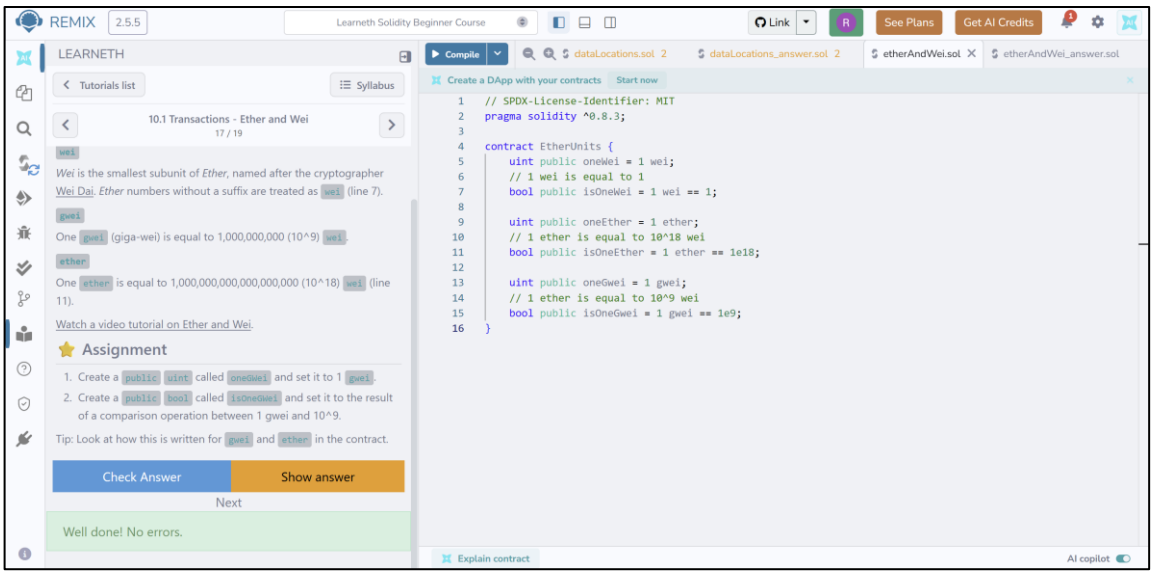


Fig 16: Transactions - Ether and Wei - assignment completed and verified in Remix.

Tutorial 17: Transactions - Gas and Gas Price

Gas consumption and gas price were examined for different operations to see how transaction cost is calculated.

Output:

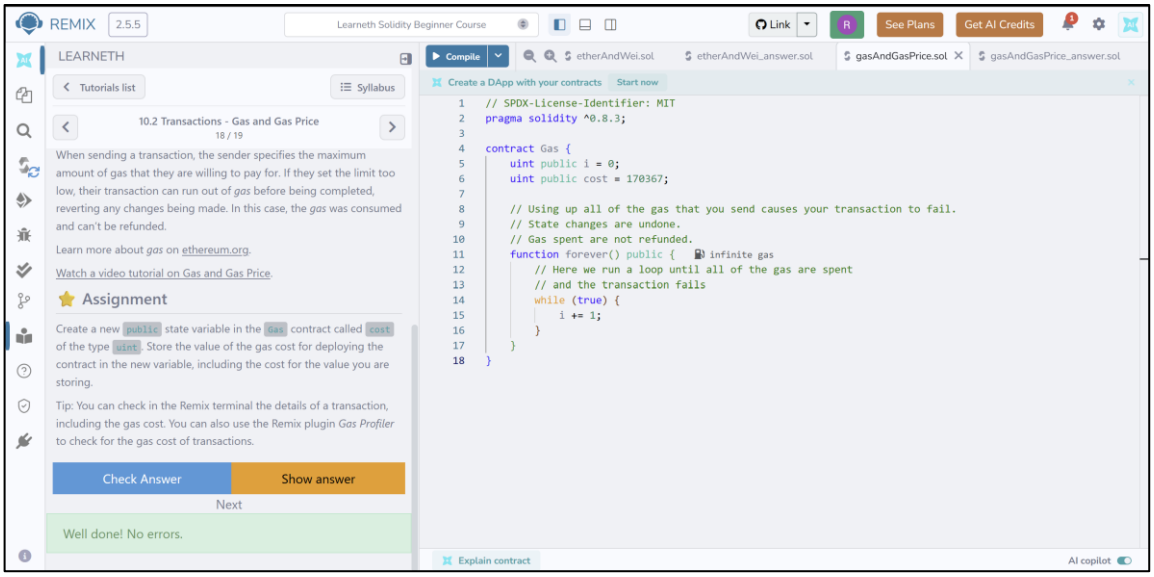


Fig 17: Transactions - Gas and Gas Price - assignment completed and verified in Remix.

Tutorial 18: Transactions - Sending Ether

A Charity contract was built with a public owner address, a payable donate() function that accepts Ether, and a withdraw() function that sends the full contract balance to the owner using call().

Output:

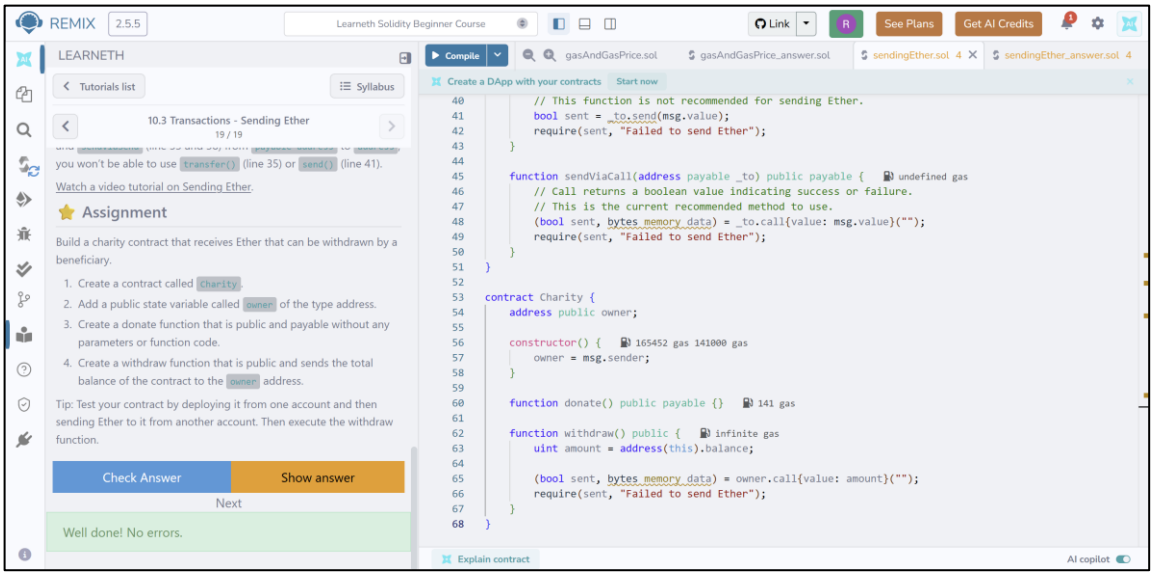


Fig 18: Transactions - Sending Ether - assignment completed and verified in Remix.

Conclusion:

Through this experiment the fundamentals of Solidity programming were explored by completing a sequence of practical assignments in the Remix IDE. The following points were observed:

1. **Language fundamentals -** Primitive data types, the distinction between state, local and global variables, and the pure and view function classifications were implemented and verified directly in the IDE.
2. **Access control -** Visibility specifiers, custom modifiers and constructors were used together to restrict sensitive operations to the contract owner, which is the standard pattern for securing a contract.

3. **Control flow and data structures -** Conditionals, loops, arrays, mappings, structs and enums were applied to model contract state, with loops in particular showing how iteration count translates directly into gas cost.
4. **Data locations -** Choosing between storage, memory and calldata was shown to be a gas optimization decision rather than a stylistic one, since storage writes are by far the most expensive operation.
5. **Transactions -** The relationship between Ether and Wei, the role of gas and gas price, and the different mechanisms for sending Ether were exercised in a working payable contract.

The hands-on practice of writing, compiling and deploying contracts on the Remix VM strengthened the understanding of how blockchain concepts translate into executable code, and provides a solid foundation for developing and managing smart contracts efficiently.